

# Rail Grade Crossing Monitoring System

---

## Code Reference & Documentation

This document covers the Zone of Interest (ZOI) stabilization module — the foundational layer of the AI-based grade crossing monitoring system. Each section contains the full code for a notebook cell followed by a plain English explanation of what the code does and why it works that way. Additional modules (vehicle detection, safety logic, alerts) will be appended to this document as the project progresses.

### Cell 1 — Imports & Environment Check

#### Code:

```
# Cell 1 - Imports & Environment Check
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Make images display inline in the notebook
%matplotlib inline

# Verify versions
print(f"OpenCV version: {cv2.__version__}")
print(f"NumPy version: {np.__version__}")
print("All imports successful!")
```

#### Explanation:

Cell 1 brings in the three core libraries the entire project depends on. OpenCV (cv2) is the main computer vision library that handles all image and video processing throughout the notebook. NumPy (np) provides the mathematical array operations that OpenCV relies on for storing and manipulating pixel data. Matplotlib (plt) handles displaying images and frames visually inside the notebook. The %matplotlib inline command tells Jupyter to render images directly in the notebook output rather than opening a separate window. The print statements at the end verify that all three libraries loaded correctly and display their version numbers as confirmation.

### Cell 2 — Load Video and Display First Frame

#### Code:

```
# Cell 2 - Load video and display first frame
video_path = "videos/Rail-Grade-Crossing-Video.mov"

# Open the video
cap = cv2.VideoCapture(video_path)

# Check it opened correctly
if not cap.isOpened():
```

```

        print("Error: Could not open video. Check the filename and path.")
    else:
        print(f"Video opened successfully!")
        print(f"Total frames: {int(cap.get(cv2.CAP_PROP_FRAME_COUNT))}")
        print(f"FPS: {cap.get(cv2.CAP_PROP_FPS)}")
        print(f"Resolution: {int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))} x {int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))}")

        # Read the first frame
        ret, frame = cap.read()

        if ret:
            # Convert BGR (OpenCV default) to RGB (matplotlib expects RGB)
            frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

            # Display it
            plt.figure(figsize=(12, 7))
            plt.imshow(frame_rgb)
            plt.title("First Frame of Video")
            plt.axis("off")
            plt.show()
        else:
            print("Error: Could not read first frame.")

    # Release the video
    cap.release()

```

### Explanation:

Cell 2 opens the video file using OpenCV's VideoCapture class, which treats the video as a sequence of individual frames that can be read one at a time. The code first checks that the file opened correctly, then reads three key properties: total frame count, frames per second (FPS), and resolution. It then reads the very first frame using `cap.read()`, which returns two values — a boolean (`ret`) confirming the read was successful, and the actual frame as a NumPy array of pixel values. Because OpenCV stores color images in BGR order (Blue, Green, Red) rather than the standard RGB order that matplotlib expects, the frame is converted using `cvtColor` before being displayed. `cap.release()` is always called at the end to properly close the video file and free up memory.

## Cell 3 — Define and Draw the ZOI Polygon

### Code:

```

# Cell 3 - Define and Draw the ZOI Polygon on the First Frame
cap = cv2.VideoCapture(video_path)
ret, frame = cap.read()
cap.release()

# Define the ZOI polygon - these are (x, y) corner points
# covering the area where the road crosses the tracks
zoi_points = np.array([
    [1500, 1100], # top-left corner
    [2250, 1100], # top-right corner
    [1750, 1340], # bottom-right corner
    [400, 1250], # bottom-left corner
], dtype=np.int32)

```

```

# Draw the polygon on the frame
frame_zoi = frame.copy()
cv2.polylines(frame_zoi, [zoi_points], isClosed=True, color=(0, 255, 0),
thickness=5)

# Label it
cv2.putText(frame_zoi, "Zone of Interest (ZOI)", (400, 980),
            cv2.FONT_HERSHEY_SIMPLEX, 2, (0, 255, 0), 4)

# Display
frame_rgb = cv2.cvtColor(frame_zoi, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(14, 8))
plt.imshow(frame_rgb)
plt.title("ZOI Polygon - First Frame")
plt.axis("off")
plt.show()

print(f"ZOI Points: {zoi_points.tolist()}")

```

### Explanation:

Cell 3 manually defines the Zone of Interest — the danger area on the railroad tracks where a stuck vehicle would be detected. The ZOI is defined as a polygon using four (x, y) coordinate points that correspond to the four corners of the crossing area between the two sets of railroad tracks. In image coordinates, X increases going right and Y increases going downward, so the top-left corner has a smaller Y value than the bottom-left corner. These coordinates were determined by examining the pixel axes in Cell 2 and identifying where the tracks intersect the road surface on both sides. OpenCV's polylines function draws the polygon as a green outline, and putText adds a readable label. The copy() call creates a duplicate of the frame so the original pixel data is not permanently modified. This manually-defined polygon is the starting point that later cells dynamically stabilize.

## Cell 4 — Detect Background Feature Points for Optical Flow

### Code:

```

# Cell 4 - Detect Background Feature Points for Optical Flow
cap = cv2.VideoCapture(video_path)
ret, frame = cap.read()
cap.release()

# Convert to grayscale - optical flow works on grayscale
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

# Detect strong corner points in the background to track
feature_params = dict(
    maxCorners=200,          # max number of points to track
    qualityLevel=0.01,       # minimum quality of points
    minDistance=30,          # minimum distance between points
    blockSize=7              # size of area considered around each point
)

feature_points = cv2.goodFeaturesToTrack(gray, **feature_params)

```

```

# Draw the detected points on the frame
frame_features = frame.copy()
for point in feature_points:
    x, y = point.ravel().astype(int)
    cv2.circle(frame_features, (x, y), 8, (0, 0, 255), -1) # red dots

# Also draw the ZOI so we can see both together
cv2.polylines(frame_features, [zoi_points], isClosed=True, color=(0, 255, 0),
thickness=5)

# Display
frame_rgb = cv2.cvtColor(frame_features, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(14, 8))
plt.imshow(frame_rgb)
plt.title(f"Feature Points Detected: {len(feature_points)}")
plt.axis("off")
plt.show()

```

### Explanation:

Cell 4 detects strong feature points in the first frame that will serve as stable background reference markers for tracking camera movement. OpenCV's `goodFeaturesToTrack` function identifies corners and edges in the image — locations where pixel intensity changes significantly in multiple directions, making them easy to locate and follow frame to frame. The parameters control how many points to find (`maxCorners=200`), the minimum corner strength required (`qualityLevel`), and how spread apart the points must be (`minDistance`). The frame is first converted to grayscale because optical flow algorithms operate on single-channel intensity values rather than full color. The detected points are drawn as red dots and land predominantly on trees, poles, signs, and crossing structures — all static background elements that are ideal for measuring camera movement.

## Cell 5 — Lucas-Kanade Optical Flow (Frame 1 to Frame 2)

### Code:

```

# Cell 5 - Lucas-Kanade Optical Flow between Frame 1 and Frame 2
cap = cv2.VideoCapture(video_path)

# Read frame 1
ret, frame1 = cap.read()
gray1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)

# Detect feature points on frame 1
feature_points = cv2.goodFeaturesToTrack(gray1, **feature_params)

# Read frame 2
ret, frame2 = cap.read()
gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

cap.release()

# Lucas-Kanade optical flow parameters
lk_params = dict(
    winSize=(21, 21),      # size of search window per pyramid level
    maxLevel=3,            # number of pyramid levels

```

```

        criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 30, 0.01)
    )

    # Track points from frame 1 to frame 2
    points2, status, error = cv2.calcOpticalFlowPyrLK(
        gray1, gray2, feature_points, None, **lk_params
    )

    # Filter only successfully tracked points
    good_old = feature_points[status == 1]
    good_new = points2[status == 1]

    # Draw the movement lines and points
    frame_flow = frame2.copy()
    for old, new in zip(good_old, good_new):
        x_old, y_old = old.ravel().astype(int)
        x_new, y_new = new.ravel().astype(int)
        cv2.line(frame_flow, (x_old, y_old), (x_new, y_new), (255, 255, 0), 2) #
        yellow = movement
        cv2.circle(frame_flow, (x_new, y_new), 6, (0, 0, 255), -1) # red =
        new position

    cv2.polylines(frame_flow, [zoi_points], isClosed=True, color=(0, 255, 0),
        thickness=5)

    frame_rgb = cv2.cvtColor(frame_flow, cv2.COLOR_BGR2RGB)
    plt.figure(figsize=(14, 8))
    plt.imshow(frame_rgb)
    plt.title(f"Optical Flow - Frame 1 to Frame 2 | Tracked Points: {len(good_new)}")
    plt.axis("off")
    plt.show()

    print(f"Points successfully tracked: {len(good_new)} / {len(feature_points)}")

```

### Explanation:

Cell 5 runs the Lucas-Kanade optical flow algorithm, which tracks how each of the 200 feature points from Frame 1 moved to a new position in Frame 2. The algorithm works by analyzing a small window of pixels around each point and searching for the best matching position in the next frame using a pyramid-based approach — the image is analyzed at multiple scales (controlled by `maxLevel`) so both large and small movements can be detected reliably. The function returns the new point positions, a status array indicating whether each point was successfully tracked (1) or lost (0), and an error value. Yellow lines connect each point's old position to its new position showing movement direction and distance. Between two consecutive frames with a stable camera the lines are extremely short because the points barely move — which is expected and confirms the camera is steady. During camera shake from a passing train, these lines would be visibly longer.

## Cell 6 — RANSAC Homography & ZOI Stabilization

### Code:

```

# Cell 6 - RANSAC Homography to Stabilize the ZOI
H, mask_ransac = cv2.findHomography(
    good_old, good_new,

```

```

        cv2.RANSAC,
        ransacReprojThreshold=3.0
    )

    # Separate inliers (stable background) from outliers (moving objects)
    inliers = good_new[mask_ransac.ravel() == 1]
    outliers = good_new[mask_ransac.ravel() == 0]

    print(f"Total tracked points : {len(good_new)}")
    print(f"Inliers (background): {len(inliers)}")
    print(f"Outliers (moving obj): {len(outliers)}")

    # Warp the ZOI polygon using the homography
    zoi_float = zoi_points.astype(np.float32).reshape(-1, 1, 2)
    zoi_stabilized = cv2.perspectiveTransform(zoi_float, H).reshape(-1,
2).astype(np.int32)

    frame_ransac = frame2.copy()

    # Original ZOI in green, stabilized ZOI in blue
    cv2.polylines(frame_ransac, [zoi_points], isClosed=True, color=(0, 255, 0),
thickness=3)
    cv2.polylines(frame_ransac, [zoi_stabilized], isClosed=True, color=(255, 0, 0),
thickness=3)

    for pt in inliers:
        cv2.circle(frame_ransac, tuple(pt.astype(int)), 6, (0, 255, 0), -1)
    for pt in outliers:
        cv2.circle(frame_ransac, tuple(pt.astype(int)), 6, (0, 0, 255), -1)

    frame_rgb = cv2.cvtColor(frame_ransac, cv2.COLOR_BGR2RGB)
    plt.figure(figsize=(14, 8))
    plt.imshow(frame_rgb)
    plt.title("RANSAC - Inliers vs Outliers | ZOI Stabilization")
    plt.axis("off")
    plt.show()

```

### Explanation:

Cell 6 applies RANSAC (Random Sample Consensus) to compute a homography matrix — a mathematical transformation that describes how the camera moved between frames. RANSAC works by repeatedly sampling random subsets of tracked point pairs and finding the transformation that explains the most points consistently. Points that fit the transformation within the `ransacReprojThreshold` pixel tolerance are called inliers (stable background) while those that do not fit are outliers (potentially moving objects). Once the homography matrix `H` is computed from the stable inlier points, OpenCV's `perspectiveTransform` function uses it to warp the ZOI polygon coordinates — mathematically adjusting its corners to compensate for camera movement. The original ZOI is drawn in green and the stabilized version in blue, with the difference between the two showing exactly how much correction was applied. All 200 points being inliers confirms a stable, motionless scene between those two frames.

## Cell 7 — Full Video Pipeline (Anchored to Frame 1)

### Code:

```

# Cell 7 - Full Video Loop with ZOI Stabilization (Anchored to Frame 1)
cap = cv2.VideoCapture(video_path)

# Read and store the first frame as a permanent anchor
ret, first_frame = cap.read()
first_gray = cv2.cvtColor(first_frame, cv2.COLOR_BGR2GRAY)
first_points = cv2.goodFeaturesToTrack(first_gray, **feature_params)

cap.set(cv2.CAP_PROP_POS_FRAMES, 0)
ret, prev_frame = cap.read()
prev_gray = first_gray.copy()

fps = cap.get(cv2.CAP_PROP_FPS)
frame_count = 0
snapshots = {}

capture_times = {
    "0s - Static crossing": 0,
    "35s - Arms coming down": 35,
    "43s - Train arrives" : 43,
    "58s - Train clearing" : 58
}

print("Processing video...")

while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame_count += 1
    current_time = frame_count / fps
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Track from FIRST frame to CURRENT frame (prevents cumulative drift)
    curr_points, status, _ = cv2.calcOpticalFlowPyrLK(
        first_gray, gray, first_points, None, **lk_params
    )

    good_first = first_points[status == 1]
    good_curr = curr_points[status == 1]

    H = None
    inlier_count = 0

    if len(good_first) >= 4:
        H, mask_ransac = cv2.findHomography(
            good_first, good_curr, cv2.RANSAC, ransacReprojThreshold=3.0
        )
        if mask_ransac is not None:
            inlier_count = int(mask_ransac.sum())

    frame_drawn = frame.copy()
    if H is not None:
        zoi_resized = zoi_points.astype(np.float32).reshape(-1, 1, 2)
        zoi_stabilized = cv2.perspectiveTransform(zoi_resized, H).reshape(-1,
2).astype(np.int32)
        cv2.polylines(frame_drawn, [zoi_stabilized], isClosed=True, color=(0, 255,
0), thickness=5)
    else:

```

```

        cv2.polylines(frame_drawn, [zoi_points], isClosed=True, color=(0, 0, 255),
thickness=5)

        cv2.putText(frame_drawn,
            f"Frame: {frame_count} | Time: {current_time:.1f}s | Inliers:
{inlier_count}",
            (50, 80), cv2.FONT_HERSHEY_SIMPLEX, 1.5, (0, 255, 0), 3)

        for label, target_sec in capture_times.items():
            if abs(current_time - target_sec) < (1 / fps) + 0.1:
                if label not in snapshots:
                    snapshots[label] = cv2.cvtColor(frame_drawn, cv2.COLOR_BGR2RGB)

cap.release()
print(f"Done! Processed {frame_count} frames.")

fig, axes = plt.subplots(2, 2, figsize=(18, 10))
axes = axes.ravel()

for i, (label, img) in enumerate(snapshots.items()):
    axes[i].imshow(img)
    axes[i].set_title(label, fontsize=12)
    axes[i].axis("off")

plt.suptitle("ZOI Stabilization - Fixed (Anchored to Frame 1)", fontsize=16)
plt.tight_layout()
plt.show()

```

### Explanation:

Cell 7 runs the complete stabilization pipeline across every frame in the video. The critical design choice here is that the homography is always computed from the first frame directly to the current frame, rather than from the previous frame to the current frame. This anchor-to-frame-one approach eliminates cumulative drift — where small errors in each frame's transformation would compound over thousands of frames, causing the ZOI to gradually wander away from the tracks. For each frame, the system tracks where the original first-frame feature points have moved in the current frame, computes a RANSAC homography from those point pairs, and applies it directly to the original zoi\_points to get the correctly stabilized polygon position. The inlier count on each frame shows how many stable background reference points were found — during the train passage this count drops slightly as the train temporarily blocks some reference points, but RANSAC uses the remaining visible inliers to maintain accurate stabilization. Snapshot frames at four key moments confirm the ZOI stays correctly positioned throughout the entire crossing event.